Université Sultan Moulay Slimane

École Nationale des Sciences Appliquées

Khouribga

Compte Rendu de TP

POO et Gestion des Exceptions

Nom : AKEBBAD

Prénom : MOHCINE

Numéro d'étudiant : 24011387

Année universitaire 2026–2027

Partie I : Programmation Orientée Objet

Exercice 1 : Classe Personne

Code source

```python
class Personne:
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age
    def afficher(self):
        print("Nom :", self.nom)
        print("Prénom :", self.prenom)
        print("Âge :", self.age)
    def modifier_age(self, nouvel_age):
        self.age = nouvel_age
    def est_majeur(self):
        return self.age >= 18
# Création de deux objets
personne1 = Personne("Akebbad", "Mohcine", 20)
personne2 = Personne("Kouti", "Oussama", 16)
# Tester les méthodes
print("Personne 1 :")
personne1.afficher()
print("Majeur :", personne1.est_majeur())
print("\nPersonne 2 :")
personne2.afficher()
print("Majeur :", personne2.est_majeur())
# Modifier l'âge de la personne 2
print("\nAprès modification de l'âge :")
personne2.modifier_age(19)
personne2.afficher()
print("Majeur :", personne2.est_majeur())
```

Résultats d'exécution

```
Personne 1 :
Nom : Akebbad
Pr□nom : Mohcine
□ge : 20
Majeur : True

Personne 2 :
Nom : Kouti
Pr□nom : Oussama
□ge : 16
```

```
Apr□s modification de l'□ge :
Nom : Kouti
Pr□nom : Oussama
□ge : 19
Majeur : True
```

Exercice 2 : Héritage Personne → Etudiant

Code source

```python
class Personne:
    def __init__(self, nom, prenom, age):
        self.nom = nom
        self.prenom = prenom
        self.age = age
    def afficher_infos(self):
        return f"Nom : {self.nom}, Prénom : {self.prenom}, Âge : {self.age}"
class Etudiant(Personne):
    def __init__(self, nom, prenom, age, numero_etudiant, filiere, moyenne):
        super().__init__(nom, prenom, age)
        self.numero_etudiant = numero_etudiant
        self.filiere = filiere
        self.moyenne = moyenne
    def est_admis(self):
        return self.moyenne >= 10
etudiant1 = Etudiant("AKEBBAD", "MOHCINE", 20, "E12345", "Informatique", 13.5)
etudiant2 = Etudiant("Id", "Sara", 22, "E67890", "Mathématiques", 9.0)
print(etudiant1.afficher_infos())
print(f"Numéro d'étudiant : {etudiant1.numero_etudiant}, Filière : {etudiant1.filiere}, Moyenne : {etudiant1.moyenne}")
print(f"Est admis ? {etudiant1.est_admis()}")
print("-" * 30)
print(etudiant2.afficher_infos())
print(f"Numéro d'étudiant : {etudiant2.numero_etudiant}, Filière : {etudiant2.filiere}, Moyenne : {etudiant2.moyenne}")
print(f"Est admis ? {etudiant2.est_admis()}")
```

Résultats d'exécution

```
Nom : AKEBBAD, Pr�nom : MOHCINE, �ge : 20
Num�ro d'�tudiant : E12345, Fili�re : Informatique, Moyenne : 13.5
Est admis ? True
------------------------------
Nom : Id, Pr�nom : Sara, �ge : 22
Num�ro d'�tudiant : E67890, Fili�re : Math�matiques, Moyenne : 9.0
Est admis ? False
```

Exercice 3 : Classe CompteBancaire

Code source

```python
class CompteBancaire:
    def __init__(self, numero_compte, nom_titulaire, solde=0.0):
        self.numero_compte = numero_compte
        self.nom_titulaire = nom_titulaire
        self.solde = solde
    def deposer(self, montant):
        if montant > 0:
            self.solde += montant
    def retirer(self, montant):
        if 0 < montant <= self.solde:
            self.solde -= montant
            return True
        return False
    def afficher_solde(self):
        return f"Compte n°{self.numero_compte} ({self.nom_titulaire}) - Solde : {self.solde} €"
compte1 = CompteBancaire("FR123", "AKEBBAD MOHCINE", 500.0)
compte2 = CompteBancaire("FR456", "Jilali", 150.0)
compte1.deposer(200.0)
compte1.retirer(100.0)
compte2.retirer(200.0)
compte2.deposer(50.0)
print(compte1.afficher_solde())
print(compte2.afficher_solde())
```

Résultats d'exécution

```
[Running] python -u "c:\Users\mohcine\Documents\DATA\tempCodeRunnerFile.python"
Compte n°FR123 (AKEBBAD MOHCINE) - Solde : 600.0 €
Compte n°FR456 (Jilali) - Solde : 200.0 €
```

Exercice 4 : Gestion d'un Parc Automobile

Code source

```python
class Voiture:
    # Classe Voiture
    def __init__(self, marque = "DACIA", modele= "RX-280", année = 2010, kilométrage = 0):
        # Constructeur de la classe
        self.marque = marque
        self.modele = modele
        self.année = année
        self.kilométrage = kilométrage
    # Méthode pour faire rouler la voiture
    def rouler(self, km):
        self.kilométrage += km
        return self.kilométrage
    # Méthode pour afficher la description
    def description(self):
        print(f"marque: {self.marque}\nmodele {self.modele}\n self.année \n kilométrage: {self.kilométrage} ")
# Classe VoitureElectrique qui hérite de Voiture
class VoitureElectrique(Voiture):
    # Autonomie initiale
    autonomie = 0
    # Méthode pour charger la voiture
    def charger(self):
        self.autonomie = 100
    # Redéfinition de la méthode description
    def description(self):
            print(f"marque: {self.marque}\nmodele {self.modele}\n self.année \n kilométrage: {self.kilométrage}\nautonomie: {self.autonomie} ")
# creating objects
v1 = Voiture()
# Faire rouler v1 de 10 km
v1.rouler(10)
# Afficher la description de v1
v1.description()
# Créer une voiture électrique
v2 =  VoitureElectrique()
# Faire rouler v2 de 30 km
v2.rouler(30)
# Charger v2
v2.charger()
# Afficher la description de v2
v2.description()
```

Résultats d'exécution

```
& C:\Users\mohcine\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/mohcine/Documents/DATA/POO-voiture.py
marque: DACIA
modele RX-280
 self.année
 kilométrage: 10
marque: DACIA
modele RX-280
 self.année
 kilométrage: 30
autonomie: 100
```

Exercice 5 : Classe Livre et Bibliothèque

Code source

```python
class Livre:
    def __init__(self, titre, auteur, annee, disponible=True):
        self.titre = titre
        self.auteur = auteur
        self.annee = annee
        self.disponible = disponible

    def description(self):
        etat = "Disponible" if self.disponible else "Emprunté"
        return f"'{self.titre}' par {self.auteur} ({self.annee}) - {etat}"

class Bibliotheque:
    def __init__(self):
        self.livres = []
    def ajouter_livre(self, livre):
        self.livres.append(livre)
    def rechercher_livre(self, titre):
        for livre in self.livres:
            if livre.titre.lower() == titre.lower():
                return livre
        return None
    def emprunter_livre(self, titre):
        livre = self.rechercher_livre(titre)
        if livre and livre.disponible:
            livre.disponible = False
            return True
        return False
    def retourner_livre(self, titre):
        livre = self.rechercher_livre(titre)
        if livre and not livre.disponible:
            livre.disponible = True
            return True
        return False
biblio = Bibliotheque()
livre1 = Livre("Les Misérables", "Victor Hugo", 1862)
livre2 = Livre("Le Petit Prince", "Antoine de Saint-Exupéry", 1943)
biblio.ajouter_livre(livre1)
biblio.ajouter_livre(livre2)
print(biblio.rechercher_livre("Le Petit Prince").description())
biblio.emprunter_livre("Le Petit Prince")
print(biblio.rechercher_livre("Le Petit Prince").description())
biblio.retourner_livre("Le Petit Prince")
print(biblio.rechercher_livre("Le Petit Prince").description())
```

Résultats d'exécution

```
[Running] python -u "c:\Users\mohcine\Documents\DATA\tempCodeRunnerFile.python"
'Le Petit Prince' par Antoine de Saint-Exup♦ry (1943) - Disponible
'Le Petit Prince' par Antoine de Saint-Exup♦ry (1943) - Emprunt♦
'Le Petit Prince' par Antoine de Saint-Exup♦ry (1943) - Disponible

[Done] exited with code=0 in 0.202 seconds
```

Partie II : Gestion des Exceptions

Exercice 1 : Division sécurisée

Code source

```python
try:
    a = float(input("Entrez le premier nombre : "))
    b = float(input("Entrez le deuxième nombre : "))

    if b == 0:
        print("Erreur : division par zéro impossible.")
    else:
        resultat = a / b
        print("Le résultat de la division est :", resultat)

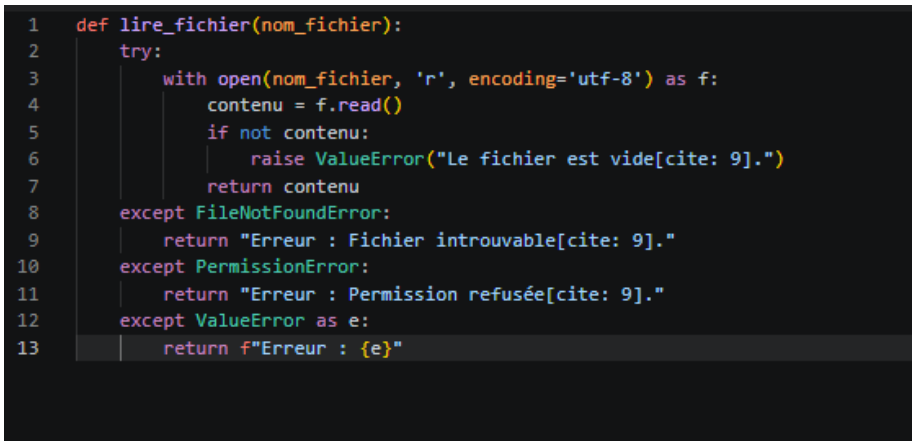
except ValueError:
    print("Erreur : veuillez saisir des nombres valides.")
```

Résultats d'exécution

```
Entrez le premier nombre : 4
Entrez le deuxième nombre : 2
Le résultat de la division est : 2.0
```

Exercice 2 : Lecture d'un fichier

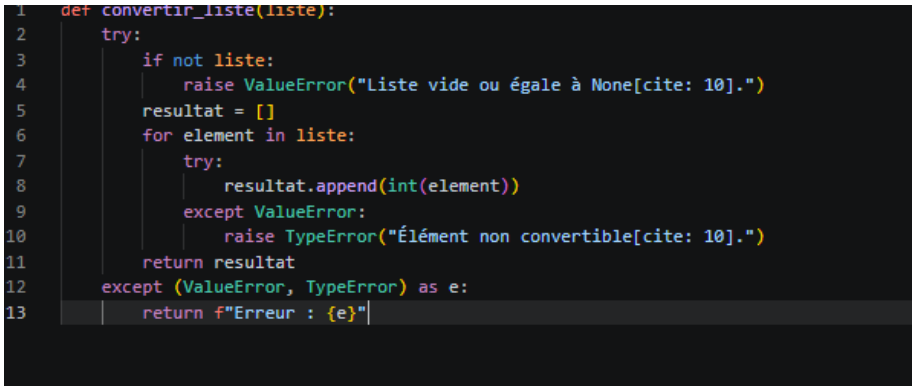
Code source



```python
def lire_fichier(nom_fichier):
    try:
        with open(nom_fichier, 'r', encoding='utf-8') as f:
            contenu = f.read()
            if not contenu:
                raise ValueError("Le fichier est vide[cite: 9].")
            return contenu
    except FileNotFoundError:
        return "Erreur : Fichier introuvable[cite: 9]."
    except PermissionError:
        return "Erreur : Permission refusée[cite: 9]."
    except ValueError as e:
        return f"Erreur : {e}"
```

Exercice 3 : Conversion de liste

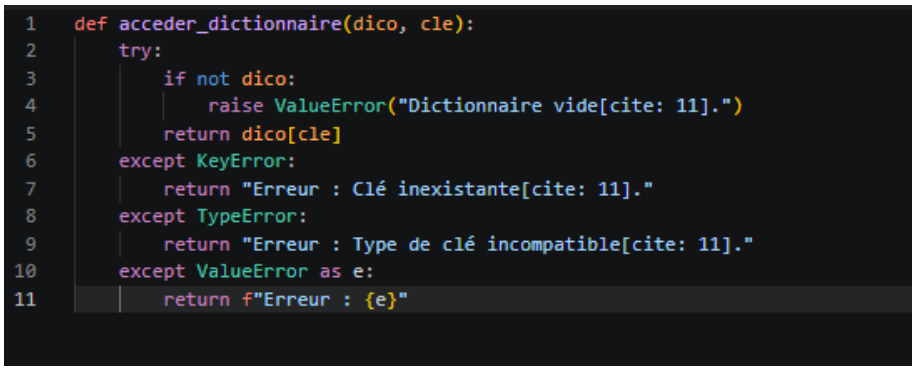
Code source



```python
def convertir_liste(liste):
    try:
        if not liste:
            raise ValueError("Liste vide ou égale à None[cite: 10].")
        resultat = []
        for element in liste:
            try:
                resultat.append(int(element))
            except ValueError:
                raise TypeError("Élément non convertible[cite: 10].")
        return resultat
    except (ValueError, TypeError) as e:
        return f"Erreur : {e}"
```

Exercice 4 : Accès à un dictionnaire

Code source



```python
def acceder_dictionnaire(dico, cle):
    try:
        if not dico:
            raise ValueError("Dictionnaire vide[cite: 11].")
        return dico[cle]
    except KeyError:
        return "Erreur : Clé inexistante[cite: 11]."
    except TypeError:
        return "Erreur : Type de clé incompatible[cite: 11]."
    except ValueError as e:
        return f"Erreur : {e}"
```

Exercice 5 : Racine carrée

Code source

```python
import math
def racine_carree(nombre=None):
    try:
        if nombre is None:
            raise ValueError("Absence de paramètre[cite: 12].")
        if not isinstance(nombre, (int, float)):
            raise TypeError("Type incorrect (liste, chaîne...)[cite: 12].")
        if nombre < 0:
            raise ValueError("Nombre négatif[cite: 12].")
        return math.sqrt(nombre)
    except (ValueError, TypeError) as e:
        return f"Erreur : {e}"
```